



Angular Avançado

Sessão 2

Acesso a dados, sub-componentes e design reativo com Signals

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 1

Recapitulação rápida antes de continuar

Projeto & anatomia

- Skyboard: repo clonado, npm start
- Angular 22: sem NgModule, sem sufixos
- bootstrapApplication + appConfig
- ng g c gera só .ts + .html
- OnPush como default

Componente com Signals

- `signal()` · `getter x()` · `set / update`
- WritableSignal **vs** Signal · `asReadonly()`
- FlightSearch com dados estáticos
- Signal Forms (só vocabulário): `form()` + `[formField]`
- Modelo Flight / Aircraft / Price

Template

- `(click)` · `[disabled]` · `{{ }}`
- `@if` · `@for ... track` · `@empty` · `@switch`
- `@default never` (exaustividade)
- Pipes: `date`, `json`, `number`
- `[class.x]` · `[style.color]`

Recap: o modelo mental de um Signal

```
signals — cheat sheet
const counter = signal(0);      // WritableSignal<number>

counter();                      // ler → 0
counter.set(5);                 // substituir
counter.update(v => v + 1);      // derivar do atual → 6

const ro = counter.asReadonly(); // Signal<number>, sem set

// objetos: nova referência, senão nada muda
basket.update(b => ({ ...b, [id]: true }));
```

Três regras

- Um signal tem sempre um valor inicial
- Ler = chamar o getter `x()` — também no template
- Objetos/arrays: `update` devolve **novo** objeto (spread)

Hoje: signals como nós de um grafo — `computed`, `resources` e `effects` ligam-nos entre si.

Porquê Signals? O que substituem

Não é só performance — é outra forma de escrever componentes

01 **Change detection preciso**

Antes: propriedade + Zone.js + dirty checking (`markForCheck`). Agora: o signal anuncia exatamente o binding que mudou.

02 **Menos RxJS para estado**

Antes: `BehaviorSubject` + `async pipe` + `combineLatest`. Agora: `signal()` + `computed()` sem subscriptions. RxJS fica para eventos e tempo.

03 **Caminho para zoneless**

Sem Zone.js a adivinhar quando algo mudou — é o próprio signal que dispara a atualização (módulo 8).

04 **Design declarativo**

Antes: `ngOnChanges`, `getters`, `subscribe` a sincronizar à mão. Agora: "Y deriva de X" — o framework mantém tudo coerente.

Em uma frase: um signal substitui a propriedade que o Angular tinha de vigiar por fora por um valor que anuncia as suas próprias mudanças. `ngOnChanges`, `markForCheck` e `BehaviorSubject`-para-estado eram remendos à volta dessa limitação.

E os lifecycle hooks?

Quase todos perdem a razão de existir — dois ficam, mais leves

Hook	Servia para	Agora
ngOnChanges	reagir a mudanças de @Input()	input() é signal → computed() / effect()
ngDoCheck	detetar mutações que o CD não via	imutabilidade + signals — o problema desaparece
ngAfterView/ContentChecked	correr a cada ciclo, sem saber se algo mudou	afterRenderEffect() (reativo) · afterEveryRender()
ngAfterView/ContentInit	aceder a @ViewChild / @ContentChild	viewChild() é signal → computed() · afterNextRender()
ngOnInit	pedir dados, derivar de inputs	fica, mas vazio na maioria: httpResource / computed nos campos
ngOnDestroy	unsubscribe, limpar libs externas	fica para libs externas; effect() limpa-se · takeUntilDestroyed() · DestroyRef

Padrão novo: constructor e inicializadores de campo para **ligar** (os cabos da casa) · signals para **reagir**. Já não se coloca código "no hook certo" — descreve-se de que depende cada valor.

Agenda de hoje

01 **Acesso a dados** HttpClient vs httpResource · estados · race conditions

02 **Sub-componentes** input() / output() / model() · content projection · two-way binding

03 **Design reativo** computed · resources · effects · semântica · signal graph

04 **Exercícios Skyboard** 3 exercícios curtos, um por parte



01

Acesso a dados

Do array estático ao backend: duas vias, uma decisão

Duas formas de falar com o backend

HttpClient

- Imperativo: **eu** peço, **eu** subscrevo, **eu** escrevo no signal
- Devolve `Observable<T>`
- Todos os verbos: GET, POST, PUT, PATCH, DELETE
- Continua a ser a via para **gravar / apagar**

httpResource

- Declarativo: descrevo o pedido em função de signals
- Entrada: signals · Saída: signals (`value`, `error`, `isLoading`, `status`)
- Re-executa sozinho quando os inputs mudam
- Pensado para **ler** dados

Skyboard mock API: <https://my-json-server.typicode.com/JoaoGoncalves/skyboard/flights> (GET only na prática · máx. 30 itens · match exato nos query params)

HttpClient: pedir, subscrever, escrever no signal

```
flight-search.ts
import { HttpClient } from '@angular/common/http';
import { inject, signal } from '@angular/core';

export class FlightSearch {
  private readonly http = inject(HttpClient);
  protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });
  protected readonly flights = signal<Flight[]>([]);

  protected search(): void {
    const { from, to } = this.filter();
    this.http.get<Flight[]>(API_URL, { params: { from, to } })
      .subscribe({
        next: (flights) => this.flights.set(flights),
        error: (err) => console.error('Error', err),
      });
  }
}
```

A reter

- `inject()` em vez de constructor injection
- `get<T>` → o JSON é assumido como `T` (sem validação)
- Assíncrono → Observable → subscribe
- params vira query string: `?from=...&to=...`
- Angular 21+: `HttpClient` já vem `providedIn: root` (`fetch`)

Os verbos do HttpClient

Método	Semântica
<code>get<T>(url, options)</code>	Obter recursos
<code>post<T>(url, body, options)</code>	Criar um recurso ou disparar processamento no servidor
<code>put<T>(url, body, options)</code>	Criar ou substituir um recurso
<code>patch<T>(url, body, options)</code>	Atualizar parcialmente (só as propriedades alteradas)
<code>delete<T>(url, options)</code>	Apagar um recurso
<code>request<T>(method, url, options)</code>	Genérico — o verbo vai como parâmetro

"Resource" na tabela = o objeto HTTP pedido/enviado. Não confundir com o Resource do Angular (próximo slide).

httpResource: o pedido como função de signals

```
flight-search.ts
import { httpResource } from '@angular/common/http';

export class FlightSearch {
  protected readonly filter = signal({ from: 'Lisbon', to: 'Porto' });

  protected readonly flightsResource = httpResource<Flight[]>(
    () => ({
      url: API_URL,
      params: { from: this.filter().from,
        to: this.filter().to },
    }),
    { defaultValue: [] },
  );

  protected readonly flights = this.flightsResource.value;
  protected readonly error = this.flightsResource.error;
  protected readonly isLoading = this.flightsResource.isLoading;

  protected search(): void { this.flightsResource.reload(); }
}
```

O que mudou

- Sem inject(HttpClient) visível
- Sem subscribe nem set
- O lambda é um computed disfarçado: lê filter() → fica a **observar** filter
- defaultValue evita undefined antes da 1.^a resposta
- reload() = pedir de novo **sem** mudar parâmetros

httpResource: desativar, escrever e tratar erro

desativar com undefined

```
httpResource<Flight[]>(() => {  
  const f = this.filter();  
  if (!f.from || !f.to) {  
    return undefined; // resource inativo  
  }  
  return { url: API_URL,  
    params: { from: f.from, to: f.to } };  
}, { defaultValue: [] });
```

value é escrevível

```
// value é WritableSignal → cópia local de trabalho  
flightsResource.value.update(fs => fs.slice(0, 5));  
// status() passa a 'local'; NÃO persiste no servidor
```

Cuidado com o erro

- Em estado de erro, `value()` **lança exceção**
- Verificar `error()` antes, ou olhar para `status()`
- GET por defeito; pode-se indicar `method/body`, mas **não** usar para gravar/apagar
- Opção `parse` (validação com Zod) → módulo 7

status() e race conditions

idle nada carregado ainda

loading 1.º pedido em curso

reloading novo pedido, já havia valor

error falhou

resolved carregado com sucesso

local value alterado via set/update

Mudança de parâmetros enquanto há pedido em curso



Em termos RxJS

- Novos params → **switchMap**: só a última resposta conta
- `reload()` com pedido em curso → **exhaustMap**: ignorado
- Por isso: ler com `resource`, gravar com `HttpClient`

Template: um @if por estado

```
flight-search.html
<button type="button" (click)="search()"
  [disabled]="!filter().from || !filter().to">Search</button>

@if (!error() && flights().length > 0) {
  <table> ...
    @for (f of flights(); track f.id) { <tr>...</tr> }
  </table>
}
@if (error()) {
  <div class="alert">Error: {{ error() | json }}</div>
}
@if (isLoading()) {
  <div>Loading...</div>
}
```

Notas

- `type="button"` evita submit + reload da página
- Guardar `flights()` atrás de `!error()`
- Alternativa: `@let s = status();` + `@switch (s)` com `@default never`
- `isLoading()` é true em loading e reloading

Exercício 1 — Skyboard ligado ao backend

~15 min · snippets: bloco E1

Tarefas

- Substituir o `search()` estático por `httpResource<Flight[]>`
- Expor `flights`, `error`, `isLoading` como campos
- Template: 3 blocos `@if` (dados / erro / loading)
- Desativar o resource se `from` ou `to` estiver vazio

Bónus

- `@let s = status();` e mostrar `s` num `@switch` com `@default never`
- Provocar um erro (URL inválida) e ver o comportamento
- Mudar `to` rapidamente 2× e observar o Network tab

Mock API faz match exato: `from=Lisbon` funciona, `from=lisbon` devolve []. Dados de teste em `db.json` do repo.